

// SOC Report

Splunk configuration file permissions issues found to be overly permissive.

Confidentiality and Integrity Improvements recommended.

Report Author: Finnian Moore

1 The Problem

During an audit of the Splunk SIEM tool, Alice (a level 2 SOC analyst) and myself (a level 1 SOC analyst), encountered an issue when attempting to search for criteria in Splunk. Attempting any search yielded no results, so we connected to the server via **ssh** to investigate. We discovered that the **config.conf** file in the **/opt/splunk/etc/system/local** directory had overly permissive permissions (**rwrxrwxrwx**). This meant that any user, regardless of authorization level, could read, write, or execute the file. This posed a critical level of risk to the Splunk system's confidentiality and integrity, as unauthorised users or threat actors could modify or corrupt the file. Any changes, made accidentally or deliberately, could potentially lead to operational issues or cause a security breach.

2 How we solved the problem

In order to fix the issue, these steps were taken:

- Firstly we identified that the **config.conf** file had permissions set to (**rwrxrwxrwx**), this is too open for such a critical configuration file. This was achieved using the **ls -l** command.

```
fstack@ubuntu:/$ find / -name config.conf 2>/dev/null
/opt/splunk/etc/system/local/config.conf
/home/fstack/Documents/config.conf
/home/fstack/.config/neofetch/config.conf
fstack@ubuntu:/$ cd /opt/splunk/etc/system/local/
fstack@ubuntu:/opt/splunk/etc/system/local$ ls -l
total 4
-rwxrwxrwx 1 root root 242 Sep 17 15:27 config.conf
fstack@ubuntu:/opt/splunk/etc/system/local$ █
```

As demonstrated in the above terminal output, the Splunk Configuration file, and its permissions are displayed as we found them in the audit. You can see the permissions second line up from the bottom, stating **rwrxrwxrwx**, which is **777** in octal notation. This is completely open, so anyone can read, write, or execute the file.

- Secondly, we used the **chmod** command to change the file permissions to **rw-r-x - - -** (or **750**)

```
fstack@ubuntu:/opt/splunk/etc/system/local$ sudo chmod 750 config.conf
[sudo] password for fstack:
fstack@ubuntu:/opt/splunk/etc/system/local$ ls -l
total 4
-rwxr-x--- 1 root root 242 Sep 17 15:27 config.conf
fstack@ubuntu:/opt/splunk/etc/system/local$
```

As you can see in the terminal output above, the **config.conf** file's permissions have been reset so that the file owner, who is root, has full read, write, and execute permissions, group has read and execute permissions, and other has no access at all.

These changes ensure that only root can modify the file, preventing the issue from occurring again. Root retains full access to ensure smooth maintenance if necessary, while authorised users in specific groups, such as SOC (Security Operations Center) employees, can read and execute the file. This setup allows for appropriate access control, ensuring that users with adequate authorisation can perform necessary actions without compromising security.

- Thirdly, we generated a baseline MD5 hash value for the file at the time of our investigation, so this can be used to monitor it's integrity in the future.

// Baseline MD5 hash: c953309e5295e29cbca8531f582d689a

```
fstack@ubuntu:/opt/splunk/etc/system/local$ sudo md5sum config.conf
[sudo] password for fstack:
c953309e5295e29cbca8531f582d689a  config.conf
fstack@ubuntu:/opt/splunk/etc/system/local$
```

As the above terminal output demonstrates, the **md5sum** command was used to generate the MD5 hash for the **config.conf** file. This is a unique string of numbers and letters that can be regenerated and referenced to know if any modifications have been made. If any authorised and deliberate changes are made in the future, a new hash should be generated and logged.

3 Recommendations

- **Enforce a more secure permission model.** This can be done by monitoring the permissions regularly to ensure they have not been changed, as they should remain what we have set them too now, which is: **rwX r-x - - - (750)**
- **Restrict access to authorised personnel.** Being aware of who should have access to this file (and all other files) is important, and appropriate access can be controlled via groups in Linux.
- **Backup and Version Control.** A policy of making regular backups for critical configuration files like this is very important, and should be adhered to. Using version control as a part of this can help track changes over time, and maintain valuable data for digital forensics in the event of an incident.
- **MD5 hash monitoring of file Integrity.** Using a hash like MD5 is a great way of ensuring files have not been compromised. Using the command **md5sum** on a file generates a string of numbers and digits, and it will always remain the same, so long as the file has not been altered. Any modification of the file, even of a single digit, will result in a completely different hash value. Regular monitoring of the hash value can be used to monitor and maintain a file's integrity. An automated check using **md5sum** can be added to the list of cron jobs, thereby minimising workload and alerting the security team in a minimal time window if an incident occurs where an unscheduled or unauthorised changes to files like the **config.conf** are made.

// Conclusion

Maintaining a high level of security vigilance over files, especially critical ones like the Splunk configuration file discussed in this report, is essential for ensuring a reliable and operational system. This can be achieved by regularly reviewing file permissions and monitoring their MD5 hash values to maintain confidentiality and integrity. Additionally, a robust backup policy combined with version control is vital for maintaining a secure system, allowing for quick and efficient recovery from incidents while providing detailed information about what occurred and when